JARVIS Live: voice, reviewed and planned

A cross-check of the attached research against this node's own measurements. Kampala, 16 October 2026.

Verdict: the research has the right shape and the wrong machine.

It describes a live voice system correctly in outline. Its numbers assume a GPU this node does not have, and its central premise is that transport is the problem. This node's own records say otherwise: the wait is the deep layer thinking, not the pipeline carrying bytes. So the plan below keeps the ideas that matter, drops the heavy stack, and builds on three speech components that are already installed here.

1. What the research gets right

- Full duplex is the right destination. Barge-in, listening while speaking, and letting the user cut in mid sentence are the difference between a voice interface and a recording.
- The dual stream idea is exactly right: spoken words and visual artifacts travel on separate paths, so a table is never read aloud as pipe characters.
- Native microphone capture matters. The Android WebView passes audio through echo cancellation and gain control, which chews the front off short utterances. Raw capture with a pre-roll buffer fixes it.
- The end state it draws is correct: self hosted, private, with our own cloned voices rather than a provider's presets.

2. What it gets wrong for this node

Every figure below was read from this machine on 16 October 2026, not assumed.

The research proposes	The truth on this node
Kokoro on an A100, or Fish Speech S2 with 12 to 24 GB of VRAM	No GPU at all. 4 CPU cores, 24 GB of RAM, no swap configured. A 4.4 billion parameter speech model would consume the whole machine and leave nothing for the agent that does the work.
Install WhisperLiveKit, Pipecat, Silero VAD, Torch	faster-whisper, CTranslate2 and ONNX Runtime are already installed in the page's own venv. Torch, Pipecat and WhisperLiveKit are not, and two of those are heavy for a gain this node cannot use.
Self hosted Fish Speech S2 for the JARVIS and Ultron voices	Both voices already exist as clones on the account in use, and the account is on a free tier. A local copy would be worse, larger, and, per its own licence line, not clearly usable in a business.
WebRTC for the media transport	WebSockets are enough. The Cloudflare tunnel carries them today. WebRTC adds a second signalling stack and saves milliseconds against a delay measured in tens of seconds.
A 300 millisecond budget for the whole turn	Unreachable and not what is wanted. The first spoken cue can land inside 300 milliseconds. The answer itself cannot, because the agent has to do the work first.
A wake word engine in the background service	Deferred. It needs a third party account and key, against the rule of no paid or card backed services. Worth revisiting when the voice is worth keeping alive in the background.

3. What our own numbers actually say

The node records every finished turn with its own timing. 478 turns over the last ten days, read from the service journal on 16 October 2026.

Measure	Result
Time to first token, all turns	median 7.3 s, best 0.4 s, worst 306.3 s

Measure	Result
Turns that reached first token in under 10 s	244 of 478 (51 per cent)
Time to completion	median 27.5 s, p90 85.5 s
Turns that took longer than 60 s	79 of 478 (17 per cent)

Split by what the turn actually was, the picture is not one distribution but two:

Turn type	Time to first token
Quick conversational turns (finished in under 20 s)	median 0.9 s
Turns that involved real work (20 s or more)	median 35.6 s
A turn started within 30 s of the previous one	median 0.9 s, and 88 per cent under 10 s
A turn started 30 s to 5 minutes after the previous one	median 23.7 s, only 42 per cent under 10 s

Two conclusions follow, and they set the whole plan. First, a conversation is already fast: the common case is under a second. Second, when the agent has to think or use a tool, the first token arrives around half a minute, and no change to the transport will move that. What it costs the user is silence, and silence is the thing to fix.

The slow turns are not a cold process. The first turn after a fresh start had a median of 7.6 s against 6.9 s for later turns in the same process, which rules that out as the cause.

4. The plan

Five phases, in order. Each one is useful on its own and leaves the text surface working. The last is the only one that touches infrastructure.

Phase	What ships	Done when	Proof
0. Freeze the baseline	The numbers in section 3 written to a file, plus a non regression checklist for the text surface (a turn, a table, the work trace, an artifact).	A rerun of the benchmark produces the same figures, and a text turn still passes every check after each later phase.	The baseline file and the checklist output.
1. Instant cue, streaming speech	Sound within a quarter second of you stopping, and the answer starting to speak before it has finished generating, sentence by sentence. No new dependencies: the node already has the interrupt path, the output gain, and the hook that splits spoken words from visual ones.	A spoken turn makes sound inside 300 ms of you stopping, and the answer begins speaking before generation completes.	Node timestamps for cue, first token and first sound, plus a recording of one turn.
2. Streaming ears on the node	A WebSocket ear: 16 kHz audio in, voice detection, and sliding window recognition that emits partial words while you speak. Built on what is installed: ONNX Runtime for voice detection, faster-whisper and CTranslate2 for words, all local.	Partial words appear on screen while you are still speaking, and the end of your speech is recognised within about half a second of you stopping.	Transcribed partials with timestamps from a real clip, and the measured real time factor on this CPU.
3. Native capture in the app	A Kotlin capture plugin: raw 16 kHz mono from the microphone, with a one second rolling buffer so the first syllable survives. Rebuilt APK.	A short word spoken the moment you tap is transcribed intact, where the WebView loses it today.	The same phrase recorded both ways, with both transcripts shown side by side.
4. Barge in, both directions	Speaking while the answer is playing stops the sound and gives you the floor. The visual state machine stays honest: idle, listening, thinking, speaking.	Speech stops within 150 ms of you starting to talk, on the phone and in the page.	Measured stop to silence, and the state timeline for the same turn.

Phase	What ships	Done when	Proof
5. Keep the fast lane warm	A persistent agent session, and a keep warm pulse at the interval the data names, so a conversational turn never queues behind a long job. Infrastructure, not interface.	Conversational turns stop having a long tail: the p90 falls from the 80 s recorded above.	The same benchmark, rerun after the change, compared with the baseline file.

5. What I am not proposing, and why

- No Pipecat. It is a large framework for a pipeline that is a fan out, not a graph. A few hundred lines of asynchronous code on the node does the same job with no new dependency, and the rule here is small single purpose builds over heavy installs.
- No WebRTC. WebSockets over the existing tunnel, binary frames for audio, JSON frames for the rest.
- No local speech model for the voices. The two cloned voices on the account work today and cost nothing. Replacing them with a local billion parameter model would be worse on this hardware.
- No wake word yet, and no background service yet. Both are worth doing once voice is worth keeping alive, and neither is worth doing before the foreground experience is good.

6. What this plan does not cover

- How the phone itself performs. Every measurement here is from the node. Audio capture quality on the handset needs the handset.
- Offline operation. The answer comes from the agent on this node, so the network stays in the path.
- Multiple users. This is one account, one owner, one device at a time.
- Recognition accuracy for Luganda and Swahili. The current fallback model is English first, and measuring the others is its own piece of work.

7. What I need from you

- Say go and I start phases 1 and 2. Neither needs anything installed: the recognition stack, the voice detection runtime and the interrupt path are already on this machine.
- Phase 3 needs a new build of the app, which means one install on your phone. I can have it ready before you need it.
- Phase 5 is the only one that touches infrastructure, and it is best done when you are not mid conversation. Name a quiet hour, or tell me to pick one.

Sources

Every figure in sections 2 and 3 was read from this node on 16 October 2026: the turn records from the service journal, machine details from the running system, and the installed package list from the page's own virtual environment. The proposals being cross checked are from the attached research document, also dated 16 October 2026. No figure here is an estimate.